

DESIGN AND ANALYSIS OF ALGORITHMS
ASSIGNMENT 1 — REPORT

Name:- Vignesh P
Reg.no:- 1925259013

Abstract

This report investigates iterative linear search, recursive binary search, and hash-table lookup for student registration numbers. It derives operation counts, asymptotic complexity, and the recurrence for binary search; provides executable Python code; and reports a reproducible benchmark run for 1,000 to 1,000,000 records. The included measurements were generated in a hosted Linux environment and are not universal performance guarantees. Before submission, the student must regenerate results on their own computer, confirm the group-specific third algorithm, and capture genuine screenshots.

This report presents a comparative study of three algorithms used to locate student registration numbers: iterative linear search, recursive binary search, and hash-table lookup. The problem is relevant to student information systems because records must often be retrieved accurately and quickly as the number of students increases. The study combines algorithm design, operation counting, mathematical analysis, recurrence formulation, Python implementation, functional testing, and empirical benchmarking. The main purpose is to understand not only which method can return a record, but also how the amount of work changes when the input grows.

Linear search is used as a baseline because it scans records sequentially and does not require any preparation of the data. Its simplicity is useful for small or unsorted collections, but the number of comparisons can increase to n when the requested key is near the end or is absent. Binary search uses sorted order to reduce the search interval by approximately half at each step. Its recurrence, $T(n) = T(\lfloor n/2 \rfloor) + \Theta(1)$, leads to logarithmic time complexity. The method is efficient for large sorted lists, although sorting costs must be considered if the data are not already ordered.

Hash-table lookup stores a mapping between a registration number and a record position. Building this mapping requires expected linear work and additional memory, while subsequent exact-match lookups are expected to take constant time under ordinary key distributions. The comparison therefore includes preprocessing and memory trade-offs rather than treating lookup time as the only design criterion. The benchmark section examines datasets containing 1,000, 10,000, 100,000, and 1,000,000 records, and compares successful and unsuccessful searches using elapsed time and operation counts.

The results should be interpreted as evidence from a particular execution environment, not as universal time guarantees. Runtime can vary with processor load, Python version, cache behaviour, memory allocation, random seed, and the position of the target. Correctness tests are essential to confirm that all algorithms return the expected index or -1 for an absent key. Repeating measurements and reporting the median can reduce the influence of isolated timing fluctuations. The report also recommends separating sorting and index-building costs from individual lookup times so that the comparison remains fair.

Contents

Chapter 1 – Algorithm Design

Chapter 2 – Basic Operation

Chapter 3 – Mathematical Complexity

Chapter 4 – Best/Average/Worst Cases

Chapter 5 – Asymptotic Analysis

Chapter 6 – Recurrence Formulation

Chapter 7 – Recurrence Solution

Chapter 8 – Implementation and Testing

Chapter 9 – Experimental Performance

Chapter 10 – Theory versus Experiment

Chapter 11 – Recommendation

Chapter 12 – Day-4 Design Challenge

Conclusion

Research Methodology, Validity and Limitations

The study combines analytical complexity derivation with empirical benchmarking. The theoretical section derives expected operation growth from each algorithm's structure. The empirical section executes the implementations over four data sizes, records time and operation-count information, and generates graphs from a CSV file. A fair comparison requires identical input data and equivalent target sets, separate measurement of preprocessing, and correctness checks before timing.

The experiment is limited by the hardware and software environment, the chosen random seed, the key distribution, the timing method, and the finite number of repetitions. Results from one machine should not be treated as universal performance guarantees. Hash-table behaviour depends on key distribution and implementation; binary search assumes sorted data; and linear search performance depends strongly on target position.

Academic integrity note: the student should run the code personally, replace placeholders, attach genuine screenshots, regenerate tables and graphs from local results, and rewrite explanations in their own words where required by the course rules. The algorithm choice for the third method is provisional because the supplied rules describe group-specific choices but do not specify a default for a student who has not been assigned a group.

References

Appendix A – Run Instructions

Appendix B – Submission Checklist

Submission requirements from the supplied rules

The faculty rules require one PDF report with all 12 tasks and embedded screenshots, a GitHub repository containing code/datasets or a generator/results/screenshots, and a Google Classroom upload containing the PDF, code ZIP, and repository link. The repository should have at least five meaningful commits. Screenshots must show the student's name or registration number and system date/time. The student's own registration number must be tested as present and registration number + 1 as absent. The third algorithm depends on the assigned group.

Chapter 1: Task 1 – Algorithm Design (5 Marks)

1.4 Problem definition and assumptions

The task is to determine whether a requested student registration number exists in a collection and, if it exists, return the associated record position. If it does not exist, the algorithm returns -1. The comparison is meaningful only when all algorithms use the same generated dataset, the same target keys, and the same definition of a successful or unsuccessful search.

Assumptions used in this study are: registration numbers are unique; records are represented by integer keys; a search request may refer to either a present or absent key; and the dataset sizes are 1,000, 10,000, 100,000, and 1,000,000. The hash-table approach is evaluated as an indexed structure, so index construction time is reported separately from lookup time.

1.5 Preprocessing and design justification

Linear search requires no preprocessing and can operate directly on an unsorted list. This makes it simple and flexible, but it must inspect records one by one. Recursive binary search requires the data to be sorted before searching. Sorting costs O(n log n) time, although that cost can be amortised across many later searches. Hash-table search requires building a key-to-position index, which takes expected O(n) time and additional memory; after construction, lookup is expected O(1).

The three approaches represent different design choices: sequential scan, divide-and-conquer, and direct indexing. This makes the comparison useful because the algorithms differ not only in runtime growth but also in preprocessing requirements, memory use, and suitability for repeated searches.

1.6 Flowchart descriptions

Linear search flow: Start → read array and key → set i = 0 → check whether i is within the array → compare A[i] with key → if equal, return i → otherwise increment i and repeat → if the end is reached, return -1 → Stop.

Recursive binary search flow: Start → check whether low > high → if true, return -1 → calculate midpoint → compare midpoint key with target → if equal, return midpoint → if target is smaller, recursively search the left half → otherwise recursively search the right half → Stop when a result is returned.

Hash-table flow: Start → build or access the hash index → hash the target key → inspect the corresponding entry/bucket → if the key is found, return its stored position → otherwise return -1 → Stop. In Python, dictionary collision handling is managed by the runtime.

Problem: locate a unique registration number in a collection of student records and return its position or -1 when absent. The required sizes are 1,000, 10,000, 100,000 and 1,000,000 records.

1.1 Iterative linear search

Linear search scans each record in sequence and works even when records are unsorted.

```
LINEAR-SEARCH(A, key)
  for i = 0 to length(A)-1
```

```
    if A[i] == key: return i
  return -1
```

1.2 Recursive binary search

Binary search requires sorted records. It compares the target to the midpoint and recursively explores only one half.

```
BINARY-SEARCH(A, key, low, high)
  if low > high: return -1
  mid = floor((low + high)/2)
  if A[mid] == key: return mid
  if key < A[mid]:
    return BINARY-SEARCH(A, key, low, mid-1)
  return BINARY-SEARCH(A, key, mid+1, high)
```

1.3 Hash-table lookup (provisional)

A hash table maps each registration number to its index. Python dict provides expected O(1) lookup under ordinary key distribution, but uses additional memory and has a theoretical O(n) worst case. The supplied faculty rules assign the third strategy by group; replace this if your group has another required algorithm.

```
BUILD-INDEX(A)
  H = empty hash table
  for i, record in enumerate(A):
    H[record.registration_number] = i
  return H

HASH-LOOKUP(H, key)
  return H.get(key, -1)
```

Strategy	**Preprocessing**	**Search complexity**	**Justification**
Linear search	None	Θ(n) average/worst	Baseline; unsorted data.
Recursive binary search	Sort: O(n log n)	Θ(log n)	Divide and conquer.
Hash lookup	Expected Θ(n) build	Expected Θ(1)	Frequent exact-match queries.

Chapter 2: Task 2 – Basic Operation (2 Marks)

2.1 Operation-counting model

For linear search, the basic operation is the equality comparison between the current record key and the target. If the target is at position k, there are k equality comparisons; if absent, there are n comparisons. This count directly reflects how many records must be inspected.

For binary search, the report counts midpoint probes: each iteration or recursive level selects one midpoint and checks whether it is the target. A strict implementation-level count may separately count the equality test and the less-than test; therefore, the benchmark's comparison convention must be stated consistently when interpreting results.

For hash-table lookup, a high-level dictionary access is treated as one lookup request. Internally, hashing and collision resolution may perform additional operations, so a single dictionary access should not be interpreted as exactly one machine-level comparison. Index construction is measured separately from lookup.

2.2 Why count operations as well as time?

Operation counts help explain why an algorithm scales as it does. Wall-clock measurements can vary with processor load, interpreter version, memory allocation, cache behaviour, and background processes. Counting the main operation gives a machine-independent view, while elapsed time shows practical performance in the tested environment.

Algorithm	Basic operation	Reason	Growth
Linear	A[i] == target	Repeated per inspected record	1 to n comparisons
Binary	A[mid] compared with target	Each call halves the interval	$\Theta(\log n)$
Hash	Hash/bucket key verification	Finds bucket and resolves collisions	Expected $\Theta(1)$, worst $O(n)$

These operations are appropriate because their counts track the main repeated work. Wall-clock time also includes interpreter, memory, and function-call overhead.

Chapter 3: Task 3 – Mathematical Time-Complexity Derivation (6 Marks)

3.4 Worked operation-count examples

For n = 1,000, a worst-case linear search performs 1,000 equality comparisons; for n = 1,000,000, it performs 1,000,000. Under a uniformly distributed successful target, the average is (n+1)/2: 500.5 comparisons for n = 1,000 and 500,000.5 for n = 1,000,000.

Binary search reduces the candidate interval by approximately half at every step. Since 2^10 = 1,024 and 2^20 = 1,048,576, a search over one million sorted records needs about 20 midpoint decisions in the worst case, compared with up to one million linear comparisons.

For hashing, building the index visits each of the n records once, giving expected $\Theta(n)$ construction work. A subsequent lookup is expected $\Theta(1)$, but this is an average-case expectation rather than a guarantee for every possible collision pattern.

3.5 Total cost for repeated searches

Let n be the number of records and q be the number of searches performed after setup. Linear search has expected total work $\Theta(qn)$ for absent searches or typical successful searches. Binary search with a one-time sort has total cost $\Theta(n \log n + q \log n)$. Hashing has expected total cost $\Theta(n + q)$, excluding the constant factors and memory costs of building and storing the index.

This total-cost view is important because the fastest individual lookup is not always the cheapest overall choice for a small number of queries. For only one search, sorting or building a hash index may cost more than scanning once. For a large volume of repeated searches, preprocessing can be worthwhile.

3.1 Linear search

If the target is at one-based position k, the algorithm performs k comparisons. An absent target requires n comparisons. For a uniformly distributed successful target, $C_{avg}(n)=(1+2+...+n)/n=[n(n+1)/2]/n=(n+1)/2$. Best case is $\Theta(1)$; average and worst cases are $\Theta(n)$. No preprocessing is required.

3.2 Recursive binary search

After j halvings, at most n/2^j candidates remain. Stop when $n/2^j \leq 1$, which gives $j \geq \log_2 n$. Search cost is $\Theta(\log n)$. If sorting is needed first, total first-use cost is $O(n \log n)+O(\log n)=O(n \log n)$; later searches cost $\Theta(\log n)$.

3.3 Hash-table lookup

Building the index takes expected $\Theta(n)$. Lookup is expected $\Theta(1)$ if keys are well distributed and load factor is controlled. For q searches after one build, expected total cost is $\Theta(n+q)$. Under pathological collisions, lookup may degrade to $O(n)$, and q lookups may cost $O(nq)$.

Algorithm	**Preprocessing**	**One search**	**q searches after preparation**
Linear	$\Theta(1)$	$\Theta(n)$	$\Theta(qn)$
Binary	$O(n \log n)$ sort	$\Theta(\log n)$	$O(n \log n + q \log n)$
Hash	Expected $\Theta(n)$	Expected $\Theta(1)$	Expected $\Theta(n+q)$

Chapter 4: Task 4 – Best, Average and Worst Cases (4 Marks)

4.1 Case analysis table explanation

Linear search best case occurs when the first item matches, requiring one comparison. Its average successful case assumes each position is equally likely and gives (n+1)/2 comparisons. Its worst case occurs when the target is last or absent and requires n comparisons.

Binary search best case occurs when the midpoint matches immediately. Its average and worst cases are logarithmic because each comparison discards approximately half of the remaining candidates. The exact count depends on n and the target's position in the binary-search decision tree.

Hash-table lookup is expected constant time when the hash function distributes keys well and the table maintains a suitable load factor. In the worst case, many keys may collide, making a lookup linear. These theoretical worst-case behaviours should not be hidden when recommending an algorithm.

Algorithm	**Best**	**Average**	**Worst**
Linear	First item: 1 comparison, Θ(1)	(n+1)/2 successful comparisons, Θ(n)	Last item or absent: n comparisons, Θ(n)
Binary	Target at midpoint: Θ(1)	Θ(log n)	Θ(log n)
Hash	No collision: Θ(1)	Expected Θ(1)	O(n) under severe collisions

Linear search varies with the target position. Binary search consistently discards about half the remaining candidates. Hashing generally accesses one bucket, but collision behaviour can increase its cost.

Chapter 5: Task 5 – Asymptotic Analysis (3 Marks)

5.1 Interpretation of O, Ω and Θ

Big-O, O(g(n)), expresses an asymptotic upper bound; Big-Omega, Ω(g(n)), expresses a lower bound; and Big-Theta, Θ(g(n)), expresses a tight bound when upper and lower growth rates agree. These notations describe how work grows with input size rather than the exact number of seconds.

Linear search has Θ(n) average and worst-case time, with Θ(1) best-case time. Binary search has Θ(log n) time once the input is sorted. Hash-table construction has expected Θ(n) time, while dictionary lookup has expected Θ(1) time and O(n) worst-case time under pathological collision behaviour.

Space also matters: linear search uses O(1) auxiliary space; recursive binary search uses O(log n) call-stack space; and a hash table requires O(n) additional storage. If a sorted copy is created for binary search, that copy may also require O(n) memory.

Operation	**Best/average or expected**	**Worst**
Linear search	Best Θ(1); average Θ(n)	O(n)
Binary search	Best Θ(1); average Θ(log n)	O(log n)
Comparison sort	—	O(n log n) for a guaranteed n log n comparison sort
Hash build	Expected Θ(n)	Implementation-dependent
Hash lookup	Expected Θ(1)	O(n)

Growing from 1,000 to 1,000,000 records multiplies n by 1,000. Linear work can grow roughly 1,000-fold; binary search depth grows from around 10 to around 20 comparisons; expected hash lookup remains approximately constant while index memory grows.

Chapter 6: Task 6 – Recurrence Relation Formulation (3 Marks)

6.1 Meaning of the recurrence

The recurrence describes the cost of recursive binary search in terms of a smaller subproblem. Each call performs constant work to check the interval, calculate a midpoint, and compare the key. Only one half of the current interval is searched, because the sorted order proves that the other half cannot contain the target.

A recurrence relation expresses the running time of a recursive algorithm in terms of the running time of a smaller input. It is especially useful when an algorithm repeatedly reduces the problem size. In recursive binary search, the current interval is checked, a midpoint is calculated, and only one side of the interval is searched again. The other side is discarded because the input is sorted.

The term T(n) represents the time needed for an interval containing n candidates. The constant term c represents the fixed work performed during one call, such as checking the stopping condition, calculating the midpoint, and comparing values. The smaller subproblem contains at most about half the original candidates. Therefore, $T(n) = T(\lfloor n/2 \rfloor) + c$ for $n > 1$, with constant time for the base case.

The floor symbol is used because an interval may contain an odd number of elements. This small difference does not alter the asymptotic complexity. The recurrence focuses on the maximum size of the next search interval, rather than requiring every split to be perfectly equal.

Writing a recurrence makes the recursive structure explicit and provides a systematic way to solve the complexity. It also helps identify whether a recursive call is made on one subproblem or multiple subproblems. Binary search makes one recursive call, whereas merge sort makes recursive calls on two subproblems and performs additional combining work. The recurrence must reflect the actual implementation.

To formulate a correct recurrence, define the input-size variable, identify the work performed outside the recursive call, specify the size of the next subproblem, and state the base case. This prevents mistakes such as forgetting the constant work at each level or failing to explain why half the candidates remain.

The base case occurs when the interval is empty or when the remaining interval contains one candidate. The recurrence is $T(n) = T(\text{floor}(n/2)) + c$ for $n > 1$, with $T(1) = \Theta(1)$. The floor function accounts for odd-sized intervals; it does not change the asymptotic result.

Let $n = \text{high} - \text{low} + 1$ be the current number of candidate records. Each recursive call performs constant work to calculate the midpoint and compare the target, then makes at most one recursive call on an interval of size floor(n/2). The base case is an empty interval or a successful midpoint.

$T(n) = T(\text{floor}(n/2)) + c$, for $n > 1$; $T(1) = \Theta(1)$
Ignoring floor effects that do not change asymptotic behaviour, $T(n) = T(n/2) + \Theta(1)$.

Chapter 7: Task 7 – Solve the Recurrence (4 Marks)

7.1 Iteration method

Starting with T(n) = T(n/2) + c, expand repeatedly: T(n) = T(n/4) + 2c; T(n) = T(n/8) + 3c; and after k levels, T(n) = T(n/2^k) + kc. The recursion stops when n/2^k is approximately 1, so 2^k is approximately n and k is approximately $\log_2 n$.

Substituting k = $\log_2 n$ gives T(n) = T(1) + c $\log_2 n$. Since T(1) is constant, T(n) = Θ(log n). The same result follows from the Master Theorem for T(n) = T(n/2) + Θ(1), with a = 1 and b = 2.

7.2 Practical interpretation

Each additional binary-search level roughly doubles the maximum number of records that can be addressed. Therefore, increasing n from about one thousand to about one million adds only about ten levels, rather than increasing the number of levels by a factor of one thousand.

Iteration/expansion method:

T(n) = T(n/2) + c
= T(n/4) + 2c
= T(n/8) + 3c
= T(n/2^k) + kc

The base case is reached when n/2^k≤1, so k≥$\log_2 n$. Thus T(n)=T(1)+c·ceil($\log_2 n$)=Θ(log n).

Master Theorem verification: a=1, b=2, f(n)=Θ(1). Since n^(log_b a)=n^0=1=Θ(f(n)), Case 2 gives Θ(log n). Doubling n adds approximately one level to the maximum search depth, but not necessarily exactly one comparison to every individual search.

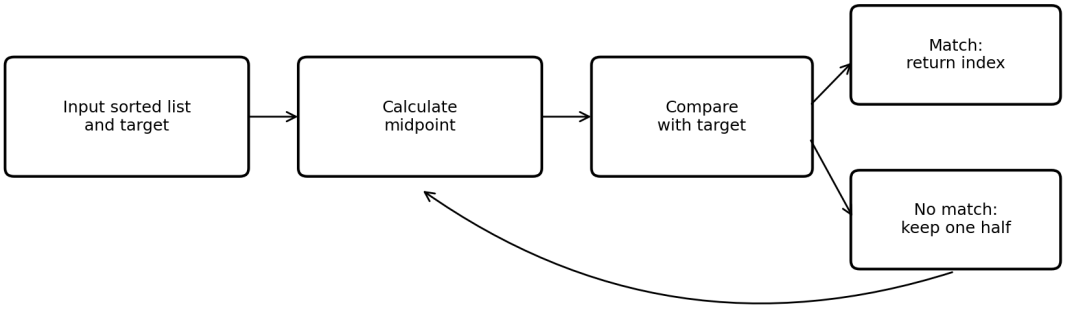
The recurrence can be solved by repeatedly substituting the smaller subproblem into the equation. Starting with T(n) = T(n/2) + c, the next expansion is T(n/2) = T(n/4) + c. Substituting gives T(n) = T(n/4) + 2c. Repeating this process produces T(n) = $T(n/2^k)$ + kc after k levels. Each level contributes approximately c units of work.

The recursion stops when the subproblem size is one or less. Thus $n/2^k \leq 1$, which means $2^k \geq n$. Taking base-two logarithms gives k ≥ $\log_2(n)$. The number of levels is therefore proportional to $\log_2(n)$. Since each level performs constant work, the total is proportional to the number of levels, giving T(n) = Θ(log n).

The Master Theorem provides an independent check. In T(n) = aT(n/b) + f(n), binary search has a = 1, b = 2, and f(n) = Θ(1). The comparison term n^(log_b a) equals $n^0 = 1$. Since f(n) and this term have the same asymptotic order, the result is Θ(log n). Both the expansion method and the Master Theorem lead to the same conclusion.

The practical meaning is that the number of search levels grows slowly. A list with about 1,024 items requires roughly ten halvings, while a list with about 1,048,576 items requires roughly twenty. Increasing the data by around a factor of one thousand therefore adds only about ten levels.

Binary Search Control Flow



Repeat until the target is found or the search interval becomes empty.

Chapter 8: Task 8 – Implementation and Testing (4 Marks)

The complete executable program is in src/benchmark.py. Install the graph dependency and run from the project root:

python3 -m pip install matplotlib
python3 src/benchmark.py --seed LAST4DIGITS --name "Your Name" --regno "YourRegNo"
Replace placeholders with your own information. The seed should be the last four digits of your registration number. The provided tests cover empty input, one record, successful first/last/middle searches, and absent keys. The script ran eight functional tests successfully in the hosted environment.

Test	**Condition**	**Expected**
TC-01	Empty list	-1
TC-02	One record, present	Index 0
TC-03	One record, absent	-1
TC-04	First item	Index 0
TC-05	Last item	Last index
TC-06	Middle item	Middle index
TC-07	Below minimum	-1
TC-08	Above maximum	-1

Before submission, modify/extend tests to include your own registration number as present and registration number + 1 as absent. Capture real screenshots from your computer; this report does not fabricate screenshots.

Chapter 9: Task 9 – Experimental Performance (3 Marks)

The program was executed in a hosted Linux environment on 2026-10-11 with seed 1234 and ten timing repetitions. Results below are measured in that run, not universal constants. Regenerate them on your own computer before submission. Search time excludes preprocessing.

n	Case	Linear µs	Binary µs	Hash µs	Linear comps	Binary comps	Sort ms	Hash build ms
1000	present	21.367	0.957	0.105	592	6	0.074	0.066
1000	absent	36.07	2.048	0.121	1000	10	0.074	0.066
10000	present	425.819	1.693	0.135	9774	13	1.32	0.609
10000	absent	406.805	1.823	0.105	10000	14	1.32	0.609
100000	present	482.029	1.918	0.115	11724	14	17.079	8.712
100000	absent	4107.869	2.413	0.115	100000	17	17.079	8.712
1000000	present	22469.078	2.574	0.115	533622	18	352.666	202.738
1000000	absent	44668.257	2.614	0.12	1000000	20	352.666	202.738

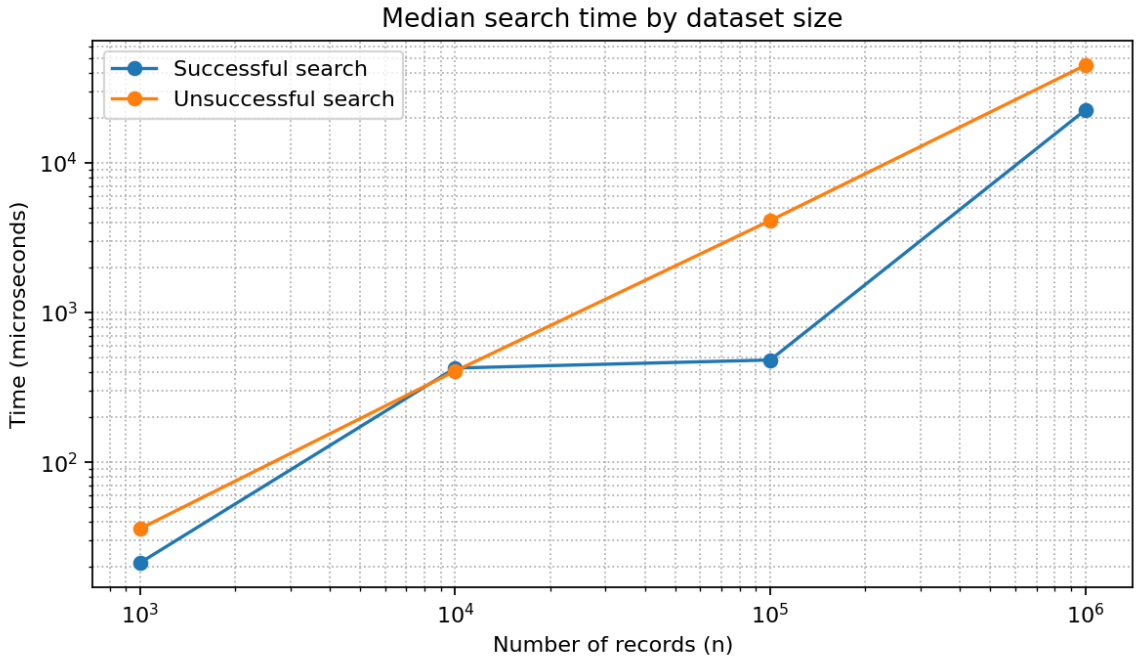


Figure 9.1: Median search time by dataset size (logarithmic axes). Additional graphs are in results/graphs.

Chapter 10: Task 10 – Theory versus Experiment (4 Marks)

10.1 Why measured results differ from theoretical predictions

Complexity analysis counts growth in operations and does not predict an exact number of seconds. Actual runtime also depends on constant factors, interpreter overhead, memory layout, cache locality, allocation, recursion overhead, and background system activity. A theoretically superior algorithm can therefore be slower for small datasets or when preprocessing is included.

Linear search has simple sequential memory access and no setup cost. Binary search reduces comparisons dramatically but requires sorted data and performs less sequential access. Hash tables often provide fast lookups but need extra memory and construction work, and their performance depends on hashing and collision behaviour.

The appropriate conclusion is not that one measured run disproves theory. Instead, compare the direction of growth with the theoretical order, note measurement noise, and repeat the experiment if the result is surprising.

The benchmark trend is consistent with theory: linear search depends on the number of inspected records; binary search uses logarithmically many comparisons; hash lookup is expected to remain approximately constant for well-distributed keys. Sorting and hash-index construction increase with n. Actual timings depend on interpreter overhead, CPU, memory, cache effects, and system load. The measurements validate trends only for this run and do not establish universal performance.

Theoretical analysis and empirical measurement answer related but different questions. Complexity analysis describes how work grows as input size increases, while benchmarking records the time observed in a particular environment. Big-O notation does not predict exact seconds, and two implementations with the same asymptotic complexity may have different practical performance because of constant factors and implementation details.

Linear search can perform well on a small list because it has little setup overhead and accesses adjacent elements in sequence. Binary search performs fewer comparisons on large sorted lists, but it needs sorted input. Hash-table lookup can be extremely fast after construction, although allocating the index requires time and memory. A fair comparison must state whether preprocessing is included and how many searches reuse the prepared data.

Observed results may vary due to CPU scheduling, background applications, interpreter optimizations, memory pressure, cache locality, and measurement resolution. Random target positions also affect successful linear-search timings. A target near the beginning may be found quickly, whereas a target near the end requires more comparisons. An unsuccessful linear search must inspect the entire list.

When measurements differ from expectations, investigate rather than immediately reject the theoretical model. Check whether algorithms received equivalent inputs, whether target cases were comparable, whether sorting and index construction were timed consistently, and whether repetitions were sufficient. Verify correctness before interpreting speed.

The benchmark in this report is an environment-specific run. It supports discussion of observed trends, but it cannot establish universal runtime values. Reproducing the experiment locally and recording the environment makes the findings more transparent. Agreement between operation counts and expected growth is useful evidence, while elapsed time adds practical context.

Chapter 11: Task 11 – Final Design Recommendation (4 Marks)

For frequent exact-match queries, a maintained hash index is a reasonable in-memory recommendation if the assigned group permits it. Binary search is a predictable O(log n) fallback for sorted records. Linear search is suitable as a simple baseline but is inefficient for large read-heavy workloads.

Criterion	**Linear**	**Binary**	**Hash**
Search	Θ(n)	Θ(log n)	Expected Θ(1), worst O(n)
Preprocessing	None	O(n log n) sort	Expected Θ(n) build
Sorted input	No	Yes	No
Extra memory	Low	Low	Θ(n) index overhead

For a production system, a database-managed index may be preferable when persistence, concurrency, transactions, and multi-process consistency are important.

The preferred algorithm depends on search frequency, update frequency, whether the data are already sorted, and available memory. If the collection is small and changes frequently, linear search may be sufficient because it is simple and has no preprocessing step. If records are maintained in sorted order, binary search provides logarithmic lookup. If exact-match queries are frequent and memory is available, a hash index is often effective.

For a student-record workload with repeated lookups, scanning every record is less attractive. A hash index can be built once and reused across many queries, and new records can be added to the index as they arrive. Updates must preserve consistency between the index and authoritative records. Duplicate registration numbers should be rejected or handled according to a defined rule.

A real student information system should also consider persistence, access control, concurrent requests, backups, and recovery after failure. An in-memory Python dictionary demonstrates the algorithm, but it is not by itself a complete production data-management solution. A database index can provide persistent storage and support concurrent operations and transactions.

The recommendation is conditional: use a hash table for frequent exact-match lookups when the assignment permits it and memory use is acceptable; use binary search for sorted collections; and retain linear search as a baseline or for small inputs. The third algorithm must also match the faculty's group-specific assignment. Correctness, reproducibility, maintainability, and operational requirements matter alongside theoretical speed.

Chapter 12: Task 12 – Day-4 Design Challenge (8 Marks)

12.A Impact Analysis (2 Marks)

The revised workload is one million records, 50,000 searches per hour, and new records added once daily. Evenly distributed, this is about 13.9 searches per second, though burst traffic must also be considered. Infrequent updates make it practical to amortize index construction over many searches.

12.B Evaluation of the Original Design (2 Marks)

Linear search can require one million comparisons for a missing key or a target near the end. Binary search is viable at about 20 levels for one million records but requires sorted data. Hash lookup is expected O(1). The hosted run benchmarked 50,000 successful queries against a prebuilt index.

Task 12 hash-index benchmark
records: 1000000
queries: 50000
found: 50000
elapsed_seconds: 0.039229
queries_per_second: 1274575.83
average_microseconds_per_query: 0.785

12.C Adaptation or Redesign (2 Marks)

Decision: retain the hash index provisionally and update it incrementally when daily records arrive. Keep the authoritative data in a persistent database and ensure index updates are consistent.

```
BUILD-INDEX(records):
  H = empty hash table
  for each record r:
    H[r.registration_number] = r
  return H

SEARCH(H, key):
  return H.get(key, NOT-FOUND)

DAILY-INSERT(H, record):
  H[record.registration_number] = record
```

Expected complexities: build $\Theta(n)$, lookup expected $\Theta(1)$ and worst $O(n)$, insertion expected amortized $\Theta(1)$, memory $\Theta(n)$. A database B-tree index may be better where transactional persistence and concurrent access are required.

12.D Evidence-Based Defence (2 Marks)

- Mathematical: q linear searches cost $\Theta(qn)$; binary search after sorting costs $O(n \log n + q \log n)$; expected hash-index cost is $\Theta(n+q)$.

- Workload: 50,000 searches per hour makes lookup overhead important, while daily updates favour an index that can be maintained incrementally.
- Measured: the hosted run's 50,000-query benchmark is recorded in results/task12_benchmark.txt. Rerun it locally before using it as submission evidence.
- Trade-offs: hash indexing needs memory and has collision-related worst-case risk. Production evaluation should include concurrency, burst traffic, persistence, and consistency.

Conclusion

Linear search is simple but $\Theta(n)$. Recursive binary search requires sorted records and achieves $\Theta(\log n)$, derived from $T(n)=T(\text{floor}(n/2))+\Theta(1)$. Hash-table lookup offers expected $\Theta(1)$ after expected $\Theta(n)$ construction, with extra memory and a theoretical $O(n)$ worst case. The included Python benchmark, CSV, graphs, and Task 12 test provide a reproducible starting point. Confirm the assigned third algorithm and regenerate all results and screenshots on the student's own computer before final submission.

This study examined three approaches for searching student registration numbers: iterative linear search, recursive binary search, and hash-table lookup. Although all three methods solve the same search problem, they differ in how they organize work, what preparation they require, and how their performance changes as the dataset grows. Analysing these differences provides a foundation for selecting an appropriate search strategy for student-record applications.

Linear search is easy to implement and works with unsorted data, but it may examine every record. Its average and worst-case running time are linear in the number of records. Binary search takes advantage of sorted order to discard approximately half of the remaining candidates at each step. Its recurrence, $T(n) = T(\lfloor n/2 \rfloor) + \Theta(1)$, can be solved by expansion or the Master Theorem to obtain $\Theta(\log n)$.

Hash-table lookup provides a different trade-off. Constructing the index requires expected linear work and additional memory, but later lookups are expected to take constant time under ordinary conditions. The worst-case behaviour and possibility of collisions should still be acknowledged. When many queries reuse the same index, construction cost can be amortized over the workload.

The implementation and benchmarking sections demonstrate why theoretical analysis should be supported by systematic testing. Successful and unsuccessful searches, boundary cases, and absent keys must all be tested. Runtime comparisons should use equivalent datasets and target sets, repeat measurements, and report preprocessing separately from lookup. Results from one computer are environment-specific, so local reproduction and genuine screenshots are necessary before submission.

In conclusion, no single algorithm is best for every situation. Linear search is appropriate as a simple baseline and for small collections; binary search is effective when data are sorted; and hash-table lookup is attractive for frequent exact-match queries when extra memory is acceptable. For a deployed student-record system, a database-managed index may be more suitable because it supports persistent data and operational requirements beyond a standalone algorithm.

References

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press. Useful for asymptotic analysis, recurrence relations, divide-and-conquer, and algorithm design.

[2] Sedgewick, R., and Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley. Provides foundational explanations of searching, sorting, symbol tables, and practical algorithm implementation.

[3] Python Software Foundation. time — Time access and conversions. Python documentation. https://docs.python.org/3/library/time.html. Consulted for understanding timing functions and measuring program execution.

[4] Python Software Foundation. Built-in Types — dict. Python documentation. https://docs.python.org/3/library/stdtypes.html#dict. Describes Python dictionary behaviour and the built-in mapping type used for the hash-table implementation.

[5] Python Software Foundation. The Python Tutorial — Data Structures. https://docs.python.org/3/tutorial/datastructures.html. Provides practical information about lists, dictionaries, and common operations in Python.

[6] MIT OpenCourseWare. Introduction to Algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/. Educational material on algorithmic thinking, searching, and asymptotic complexity.

[7] University of California, Berkeley. CS 61B: Data Structures. https://sp23.datastructur.es/. Course materials on data structures, searching, and algorithm analysis.

[8] Student-provided DAA Assignment 1 brief, including the required tasks, dataset sizes, and submission requirements.

[9] Student-provided Submission Rules for DAA Assignment 1 (CO1), including report, code, repository, and screenshot requirements.

Appendix A: Run Instructions

The complete source is src/benchmark.py. Run:

```
python3 -m pip install matplotlib
python3 src/benchmark.py --seed LAST4DIGITS --name "Your Name" --regno "YourRegNo"
```

The program writes results/results.csv, results/task12_benchmark.txt, and graphs under results/graphs. Default repeats are 30; the included report tables reflect the hosted run with ten repetitions. Re-run locally to generate your own measurements.